GitHub repository: https://github.com/std7777/shell

CppShell Mid Progress Review

1 Project Title and Team Details

Project title: Design and Implementation of a POSIX-Compatible Shell Using C++
Student: Vachhani Priyanshi Hetalkumar
Registration number: 24BCI0232
Course: Operating Systems
Guide or faculty: SUCHITHRA J
Department: CSE with Information Security
Review date: 7 October 2026

2 Introduction and Background

A command-line shell accepts textual commands and connects them to operating-system services. CppShell is a C++23 project that exposes this flow through a compact interactive implementation. The current version supports command dispatch, built-in commands, executable discovery through PATH, external command execution, and persistent directory navigation.

The project demonstrates environment-variable handling, executable permissions, filesystem state, standard input and output, and the distinction between operations performed inside the shell process and programs launched outside it. Planned stages extend these ideas to child processes, file descriptors, redirection, and inter-process communication through pipes.

3 Problem Statement

The project addresses how a shell interprets user input, distinguishes built-in commands from external programs, locates executables, maintains process state such as the current working directory, and reports errors without ending the interactive session. The final scope also includes direct process creation, stream redirection, and pipelines.

The target is a defined subset of POSIX shell behavior. Shell scripting, job control, background execution, wildcard expansion, command substitution, aliases, functions, and persistent command history remain outside the initial scope.

4 Objectives

- Provide a persistent prompt that reads and dispatches commands until the user exits.
- Implement echo, exit, type, pwd, and cd, including absolute, relative, and home-directory navigation.
- Locate external executables through PATH and run them with command-line arguments.
- Add direct process execution, output and error redirection, and multi-command pipelines.
- Manage child processes and file descriptors safely and recover from command errors.

5 Methodology and System Design

Development proceeds feature by feature. The original single-file program has been divided into an entry point, shared lookup utilities, a command dispatcher, and one handler for each built-in. The dispatcher maps command names to functions, allowing built-ins to run in the CppShell process while external commands follow a separate lookup and execution path.

Current execution flow: main.cpp reads PATH and stores its valid directories. The REPL prints a prompt and reads a line. The dispatcher separates the command name from the remaining input. Registered built-ins execute directly. Other names are searched in PATH and, if found, the complete input line is passed to std::system().

Current component	Responsibility
main.cpp	Initializes stream flushing, reads PATH, prepares executable lookup, and starts the REPL.
command_handler.hpp	Registers five built-ins, splits input, dispatches commands, and controls the prompt loop.
common/common.hpp	Stores built-in names and PATH directories and validates executable candidates.
builtins/echo.hpp	Prints the input text.
builtins/exit.hpp	Returns the control value that ends the REPL.
builtins/type.hpp	Identifies built-ins or prints the first executable path found through PATH.
builtins/pwd.hpp	Reads and prints the shell process's current working directory.
builtins/cd.hpp	Changes the shell process's working directory and expands a leading tilde through HOME.

Operating-system behavior: pwd and cd run as built-ins inside CppShell. This is necessary for cd because changing the directory of a separate child process would not change the shell's own working directory. The implemented cd uses std::filesystem::current_path(), so successful changes remain active for later commands.

Current execution boundary: std::system() delegates external execution and waiting to the host command interpreter. CppShell does not yet create child processes with fork(), replace a child with exec(), or collect status with waitpid(). It also does not yet create or connect file descriptors for redirection and pipelines.

Technologies and concepts: C++23, CMake, Git, STL containers and function objects, std::filesystem, PATH and HOME environment variables, executable-permission checks through access(..., X_OK), standard streams, and persistent process state.

6 Work Completed and Current Progress

Feature	Current state
Interactive prompt and dispatch	Implemented for ordinary command input, built-in lookup, external lookup, and unavailable-command diagnostics.
echo, exit, and type	Implemented as separate handlers. type recognizes all five current built-ins and resolves external executables through PATH.
PATH executable resolution	Implemented in directory order with existence, regular-file, and executable-permission checks.
External execution	Implemented through std::system() after PATH lookup. Direct fork/exec/wait handling remains pending.
pwd	Implemented using std::filesystem::current_path().
cd	Implemented for filesystem paths; relative and absolute paths are handled by current_path(), and a leading tilde is expanded from HOME.
Parser, redirection, pipelines	A first-space command split exists. Structured parsing, native redirection, and pipeline management remain pending.

Current status: the project now completes the planned basic built-in set: echo, exit, type, pwd, and cd. PATH-based executable discovery and delegated external execution are also present. Directory changes occur within the long-running shell process and therefore affect subsequent pwd and external commands.

Current limitations: input is still split only at the first space. Empty input and end-of-file are not handled explicitly. cd accesses its first input character before checking whether the input is empty, and its error message does not distinguish missing paths from permission or other filesystem errors. A nonzero result returned by std::system() is currently treated as a signal to end the REPL.

Working Model Demonstration

The terminal screenshots below record the working model presented on 6 October 2026. The Kali prompt is visible before CppShell starts. After `./build/shell` is executed, CppShell uses its own simple `$` prompt.

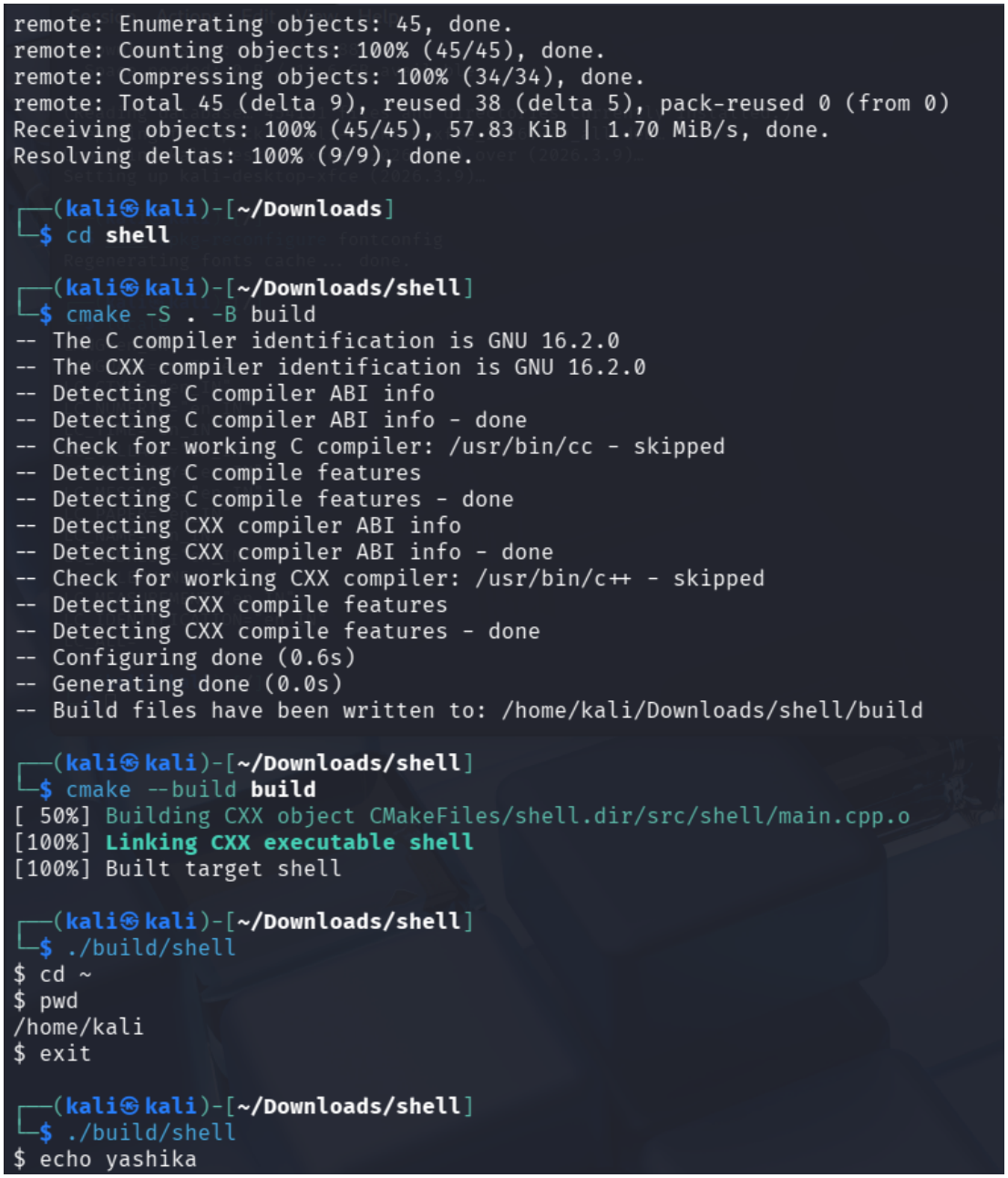


Figure 1 Configuring the project with CMake, compiling the target, creating the shell executable, launching it from the terminal, and confirming cd, pwd, exit, and echo behavior.

Built-in and External Command Demonstration

This run shows built-in output, an external program found through PATH, working-directory changes that persist inside CppShell, and diagnostics for unavailable commands.



Figure 2 Running echo, the external cat program, cd, pwd, home-directory navigation, exit, and unavailable-command handling.

Executable Resolution and Filesystem Commands

The two sessions distinguish host-shell output from commands entered after CppShell starts. They demonstrate type resolution and external filesystem utilities executed through CppShell.

```
┌──(kali㉿kali)-[~/Downloads/shell]
└─$ ls
build  CMakeLists.txt  LICENSE  src

┌──(kali㉿kali)-[~/Downloads/shell]
└─$ mkdir joy

┌──(kali㉿kali)-[~/Downloads/shell]
└─$ rm -f joy
rm: cannot remove 'joy': Is a directory

┌──(kali㉿kali)-[~/Downloads/shell]
└─$ rm -rf joy

┌──(kali㉿kali)-[~/Downloads/shell]
└─$ type cd
cd is a shell builtin

┌──(kali㉿kali)-[~/Downloads/shell]
└─$ mkdir joy

┌──(kali㉿kali)-[~/Downloads/shell]
└─$ ./build/shell
$ rm -rf joy
$ ls
build  CMakeLists.txt  LICENSE  src
$ exit

┌──(kali㉿kali)-[~/Downloads/shell]
└─$ type ls
ls is an alias for ls --color=auto

┌──(kali㉿kali)-[~/Downloads/shell]
└─$ type cat
cat is /usr/bin/cat

┌──(kali㉿kali)-[~/Downloads/shell]
└─$ jobs

┌──(kali㉿kali)-[~/Downloads/shell]
└─$ cd /
```

Figure 3 type identifies cd as a built-in and resolves cat through PATH; rm and ls run as external programs.

```
┌──(kali㉿kali)-[~/Downloads/shell]
└─$ where

┌──(kali㉿kali)-[~/Downloads/shell]
└─$ where cat
/usr/bin/cat
/bin/cat

┌──(kali㉿kali)-[~/Downloads/shell]
└─$ ./build/shell
$ ls
build  CMakeLists.txt  LICENSE  src
$ exit

┌──(kali㉿kali)-[~/Downloads/shell]
└─$ where disko
disko not found

┌──(kali㉿kali)-[~/Downloads/shell]
└─$ ./build/shell
$ touch filename
$ ls
build  CMakeLists.txt  filename  LICENSE  src
$ rm filename
$ ls
build  CMakeLists.txt  LICENSE  src
$ mkdir joy
$ ls
build  CMakeLists.txt  joy  LICENSE  src
$ rm joy
rm: cannot remove 'joy': Is a directory

┌──(kali㉿kali)-[~/Downloads/shell]
└─$ rm joy
rm: cannot remove 'joy': Is a directory

┌──(kali㉿kali)-[~/Downloads/shell]
└─$ rm -rf joy

┌──(kali㉿kali)-[~/Downloads/shell]
└─$ ls
build  CMakeLists.txt  LICENSE  src
```

Figure 4 touch, ls, rm, mkdir, and rm -rf demonstrate external programs with arguments and visible filesystem effects.

Directory Navigation and Error Cases

These sessions show successful relative and absolute directory changes, home-directory expansion, persistent pwd results, type recognition, and current cd error messages.

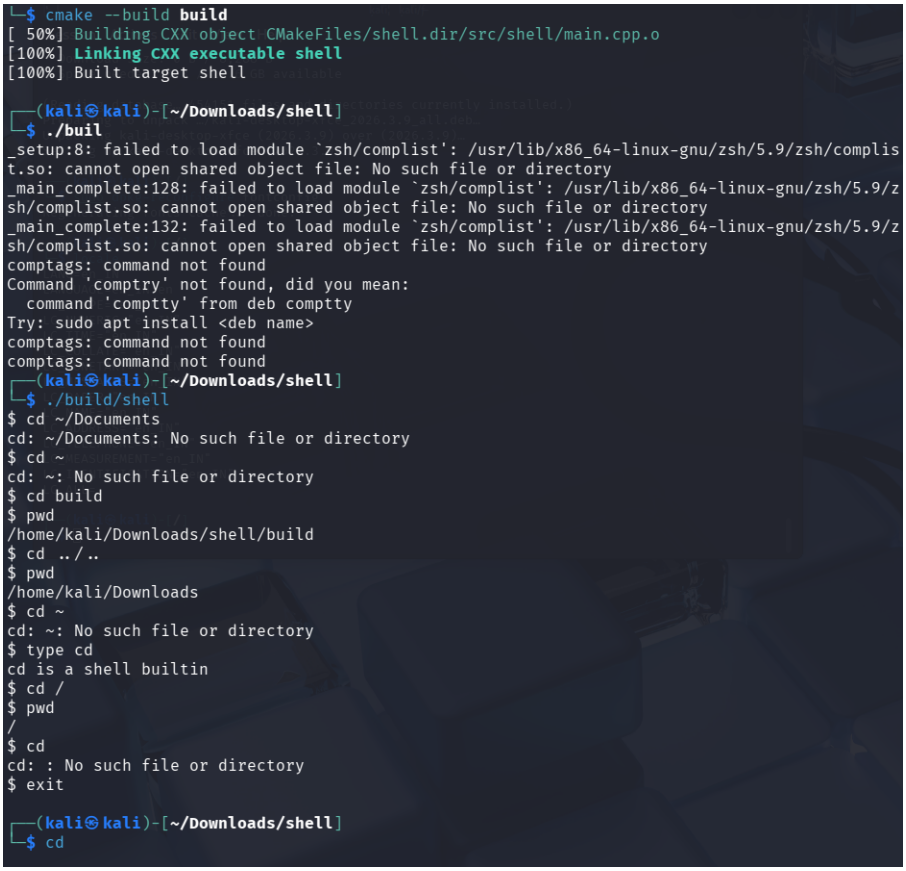


Figure 5 Rebuilding and launching CppShell, then exercising invalid paths, relative navigation, parent navigation, type cd, root navigation, and a missing cd argument.

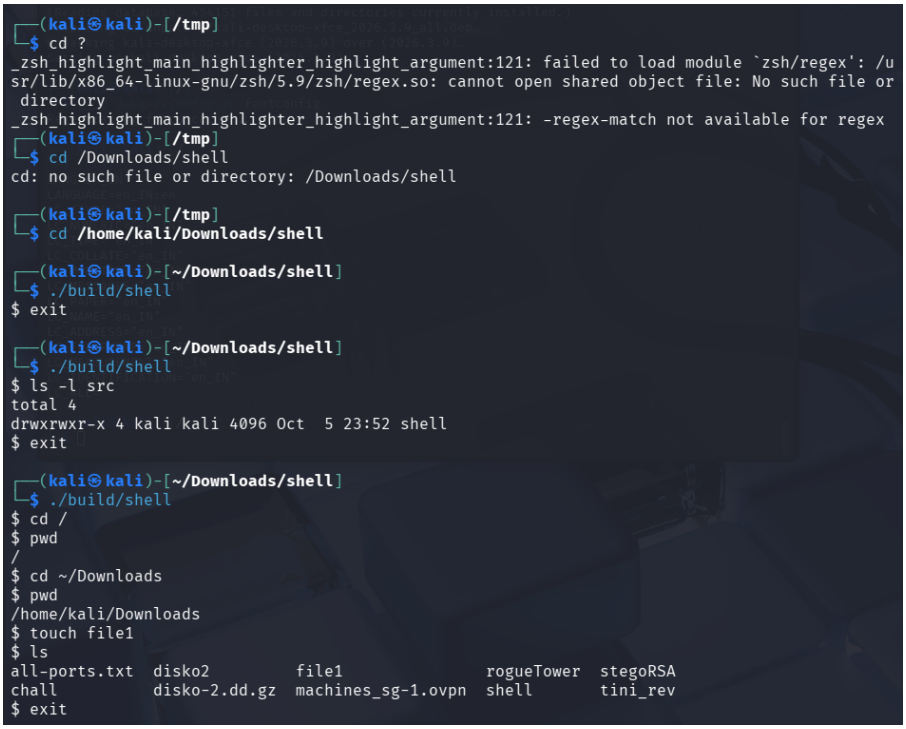


Figure 6 Navigating by absolute path, root path, and home-relative path, followed by pwd and external touch and ls commands.

7 Last Contributions and Recent Progress

The latest development phase added filesystem navigation and connected the new commands to the existing built-in registry. The project history records these milestones:

Recorded period	Development milestone
20 February 2026	Added the prompt, user input, and unavailable-command handling.
27 to 28 February 2026	Added the repeated prompt, exit, echo, and built-in identification through type.
7 March 2026	Extended type to search executable programs through PATH.
25 to 26 August 2026	Separated the program into modules and connected PATH lookup to external execution.
5 October 2026	Added pwd and cd, registered both built-ins, and added leading-tilde expansion through HOME.

The main new result since the previous review is persistent directory navigation. pwd exposes the current process state, while cd changes that state for later commands. Home-directory expansion also connects shell syntax to the HOME environment variable.

8 Remaining Work and Validation

The following features and corrections remain within the defined project scope:

- **Reliable input handling:** handle blank input, leading whitespace, missing cd arguments, and end-of-file safely.
- **Filesystem error reporting:** preserve the reason reported by the operating system so cd can distinguish a missing path, permission failure, and other errors.
- **Structured command parsing:** tokenize command names, arguments, redirection operators, filenames, and pipeline separators before allocating operating-system resources.
- **Direct external execution:** replace std::system() with fork(), an exec-family operation, and waitpid(), and pass an explicit argument vector to the child program.
- **Exit-status handling:** separate the exit built-in from external-program status so a failed child command does not terminate the shell.
- **Output and error redirection:** implement overwrite and append modes for standard output and standard error using open(), dup2(), and close().
- **Built-in redirection:** redirect supported built-ins while restoring the parent shell's standard streams after execution.
- **Pipelines:** implement two-command and multi-command pipelines with pipe(), including supported built-ins as pipeline stages.
- **Resource management:** close unused descriptors, wait for every child, and handle partial failures without leaks, unreaped processes, or deadlocks.
- **Prompt and environment improvements:** show useful location information in the prompt and make environment-dependent behavior update safely during the session.

Validation still required: check PATH ordering and executable permissions; empty input and end-of-file; valid and invalid cd targets; relative, absolute, and home paths; failed external programs; overwrite and append modes; stream restoration; multi-stage pipelines; large pipeline input; child reaping; and descriptor cleanup.

Documentation: add build and usage instructions, supported command examples, known limitations, and repeatable checks for each implemented feature.

Expected final outcome: a modular C++ shell that executes supported built-ins and external programs, preserves working-directory state, redirects standard output and error, and coordinates multi-command pipelines. The result will demonstrate process creation, program replacement, synchronization, file-descriptor management, filesystem state, and inter-process communication.